An Open IoT Framework Based on Microservices Architecture

Long Sun*[1], Yan Li[2], Raheel Ahmed Memon[3,4]

[1] College of Computer Science, Sichuan University, Chengdu 610064, China.
[2] College of manufacturing Science and Engineering, Sichuan University, Chengdu 610064, China.
[3] School of Computer Science, University of Engineering Science and Technology, Chengdu 610064, China
[4] Department of Computer Science, Sukkur Institute of Business Administration, Sukkur, Sindh, Pakistan
* The corresponding author

Abstract: With the continuous development and evolvement of Internet of Things (IoT), monolithic application becomes much larger in scale and even more complex in structure. This leads to poor scalability, extensibility and maintainability. In response to those challenges, microservice architecture has been introduced in the field of IoT application, due to its flexibility, lightweight and loose coupling. However, the existing IoT framework of microservice mainly focus on a specific domain, therefore, this greatly limits its application. In this paper, we propose a general microservice system framework for the IoT application, which is a better scalable, extendable and maintainable architecture. We introduce its system design and related microservices, and emphasize on core service and device communication from service layer to physical layer. It has better capacity to support interoperability and accommodate heterogeneous objects. In addition, this framework can easily achieve more application integration such as automation, intelligence, Geo service and Big Data.

Keywords: Internet of things; IoT; micro service; software architecture

I. INTRODUCTION

The term IoT (Internet of things) is not very new, there have been several different names and one of such names is "Internet of Infinite Things", the dreamworld which everything may communicate with each other. Today, we have a large number of physical entities interconnected and integrated into the information space to interchange generated data using communication technologies. Previously a number of research work [3-4] focused on the connectivity challenges. However, with newly developed systems, the existing structure has given birth to several new research challenges. With the development of technology, the IoT applications [5-11] have started to consider how to integrate the existing network facilities and the openness and scalability of the system design. Compared with existing services, such as telecommunication services or Internet applications, IoT service faces with new situation. The huge information collected from perception layer (involving a large number of sensors) is further processed and delivered to different applications according to the requirements, then is used to trigger the corresponding collaborative business system. As the IoT technology is widely used in daily life, the events produced by sensors and objects are becoming astronomical. The task is enormous and immense services response to the requests. IoT services system should coordinate man-

Received: Jun. 08,2016
Revised: Oct. 19. 2016
Editor: Mugen Peng

In this paper, a generic comparison between two architectures has been discussed. Based on these foundations, the authors have proposed a microservice system framework for the IoT.

power and business process to quickly interact with the physical entities across the business domains, even across the organizations based on the processing and integrating of distributed multisource sensing information. Because of the variety of business process, personal and physical entities involved, physical world continuously changes, IoT services are characterized by real time changes. It brings new technical challenges to IoT services such as: heterogeneity of hardware, network and operating system, interoperability between applications and services, fusion of massive heterogeneous data, scalability and continuous integration.

Heterogeneity of hardware, network and operating system. Available IoT devices (sensors and actuators) are vendors specific where the communication protocols and data exchange formats vary from device to device. The complexity increases exponentially when simple devices are integrated to form a complex network and it is difficult to provide a unified solution for IoT application.

Interoperability between applications and services. In the IoT application, many actors comprising human and nonhuman objects and many systems are designed for specific applications, adopting specific data standards and communication platforms. Different platforms lead to great inconvenience for achieving interoperability and mutual communication in IoT applications.

Fusion of massive heterogeneous data. While data collected in the IoT generates billion pieces of information with hundreds of data formats, it's hard to classify vast amount of raw data by a complex module in traditional architecture, and it also easily leads to system overload if a large number of redundant data directly goes to the application layer.

Scalability and continuous integration. IoT aims to keep on growing all the time and it has generated a lot of complex code. Developers with different technological skills often concern differently and everyone has their own limitations. So a good and independent architecture can offer a mechanism that the programmers can enjoy the freedom of coding with simplified integration

In a nutshell, with the development of IoT technology, traditional architecture can't meet the requirements of heterogeneous, interoperable, customizable and scalable systems. To deal with above challenges, we put forward an open IoT framework by decomposing IoT system into microservices to perform different kinds of tasks. By using message driven and registry/discovery mechanism in core service, the framework can easily extend, evolve and integrate third party applications to support interoperability and scalability. Moreover, the system uses device plugins to shield the differences of hardware facilities in order to support more heterogeneous platforms. In particular, as integrated with a series of microservices, the framework delivers strong mechanism for fusing heterogeneous data through hierarchical preprocessing mass sensors data.

The paper is organized as follows: Section 2 introduces the related research about the architecture of monolithic and microservice of IoT application. Section 3 deals with the suggested microservice IoT system architecture. Section 4 presents how to implement the system and makes a comparison among the related IoT system. Finally, Section 5 presents a conclusion on the advantages and the limits of the microservice IoT system, and the future work.

II. Related Work

The previous research of IoT system mainly includes two categories: monolithic and microservice architecture.

2.1 Monolithic architecture

In the monolithic architecture [1] software system is deployed as a single solution, in which functionally distinguishable aspects are all interwoven [2]. The natural advantages of monolithic architecture are module independent, uniform standards and technology stack. Many researches offer solutions from perspective of monolithic architecture. Earlier IoT studies mainly focus on hardware network

and low-level software technology [3]. For instance, Wireless Sensor Networks (WSN) [4] is a major approach of IoT, but it has limited capacity and weak scalablity. To address these issues, some researches like ubiSOAP [5] WoT/SDN [6] focus on high-level IoT application programming, which uses standard middleware to implement a layered communication. However, these systems are low reusability and portability. TinySOA [7], Servilla [8] and a series of typical Service Oriented Architectures (SOA) [9] have been applied in different applica-tions. To improve the scalablity of SOA, an Event-Driven SOA (EDSOA) IoT technology [10] has also been put forward. In addition, OpenIoT [11] expands the concept of SOA, and implements Web of Things (WoT).

However, with the increasing of the system functions, the IoT becomes more and more complex in the distributed environment. Monolithic architecture has some inevitable defects. First, the entire system is a united application; only multiple deployments can improve the system performance, while the overloaded functions create bottleneck, which is a waste of computing resources. Second, in the case of the change and evolution of the system, a change in a function may affect other functions due to high dependencies. This also brings complexity for re-deployment, maintenance and continuous integration. Finally, the whole system uses a sole technology stack and development standards, which in turn limits the methods to solve the problem of physical heterogeneity.

2.2 Microservice architecture

To overcome the drawbacks of monolithic architecture, many researchers started to adopt microservice architecture [12]. Micro service architecture is a new system software design pattern. It suggests that a single large and complex application should be divided into small manageable groups, where each group deals with the related services. According to its specific business responsibilities, each micro service is dedicated for a single business function. Therefore, independent services can be easily deployed and released internally to the production environment in isolation and the modification of services would not affect the whole system. Any applicable tools and languages can quickly realize a specific service. Compared with the traditional monolithic architecture, the microservice architecture has obvious advantages, such as complexity under control, independent deployment, more choices for technology stack and fault tolerance, which will facilitate the development of IoT applications on large scale.

Many researchers begin to provide IoT solution using microservice architecture. For example, Vresk and Tomislav [13] present a microservice based architecture focusing on connecting with heterogeneous devices, where the system confines to data model aspect. Krylovskiy et al. [14] discusses how to apply microservice architecture to design a Smart City IoT platform. Bak et al [15] presents three cloud microservices: contextual triggering microservice, visualization microservice, and anomaly detection and root cause microservice, to accelerate and facilitate the development of context and location based applications. However, the above mentioned microservice IoT systems only consider a specific application. Therefore, it is necessary to design a more generic and open framework in IoT system.

III. System Architecture

3.1 System design

The design of a new generation of IoT framework considers the integration and reusability of existing information service system with high cohesion and loose coupling in open and scalable platform design. The core idea of the design is adopting the concept of microservice architecture, based on the analyse of the existing IoT system, reconstructing all the business functions of the system by decoupling them into independent and specific services. The design is using lightweight communication mechanism to interact between services with a

minimal overload.

The proposed design uses a central service named as core service to control and coordinate the other microservice in the system; it is also used to bootstraps one or more tenant engines, which handle most of the other processing logic. The system adopts REST design pattern to facilitate the access to object data and to support interoperability with other network. Core communication service encompasses the communication with the objects nodes between the microservices and external networks. It is provided by a broker micro service to provide asynchronous communication. In the development of the communication service, we adopt REST to communicate with client applications.

Apart from using REST interfaces to interact with microservices, the system also implements publish subscribe model to notify the clients about the interest events from their subscribed topics, where all the subscribed topics of a client would push the upcoming event notification on client side application. From the core service broker, a client can subscribe multiple interest topics in order to receive notifications from multiple services.

Fig 1 shows the architecture of proposed microservice IoT system. The overall system is composed of different micro services that are wired together to provide a complete IoT platform. The design is consists of eight micro services and one core service coordinating with all, the microservices are as follows: Geo, Security, Tenant, Devices, Bigdata, Automation, AI and Application. The system is not limited to these services; its design is flexible which can be modified or extended according to the applications requirements.

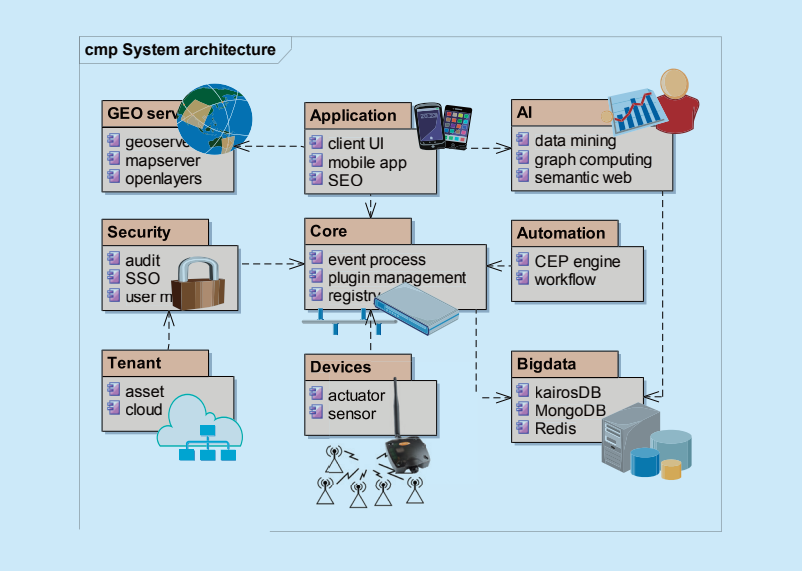

Fig.1 *The architecture of the proposed IoT framework*

1. *Geo Microservice*: The service is designed for organize devices by spatial correlation in order that the events of the devices can be gathered into groups in this way, such as location-aware devices. A Geo service provides GIS layer to implement map API for render location data on it. The service provides REST API to invoke map UI library and renders a layer or creates a layer dynamically.

2. *Security Microservice*: The service provides user/group/role management, authentication and authorization, access control, single sign-on and federation, identity governance and administration. It also supports security monitoring, reporting and auditing. User credentials are required for accessing the REST services. When performing create/update operations on entities, the access control policies of the authenticated user are stored to indicate which action can be performed by the user.

3. *Tenant Microservice*: Based on the multitenant design, the service provides support for multiple IoT applications by a single core service instance. An independent data store and processing pipeline is owned by each tenant, thus data and processing will not blended between tenants. Most components are also designed as multitenant enabled. This supports logic isolation of processing and data from different tenants.

4. *Device Microservice*: The device service manages various kinds of device plugins, and implements different communication protocols with low level hardware of how to collect data from sensors and execute command to actuators. This service provides interface for core service to invoke plugins callback. It uses metadata that contains extend context to specific device components. Based on speci-

fications which contain device metadata and unique information like hardware identification, we can assign device plugins associated with physical objects. More details will be introduced in the following subsection.

5. *Bigdata Microservice*: The Bigdata service provides support for many kinds of scalable persistence to store data. It interacts with storage implementations by a consistent encapsulated API set. By this way, it is easy to add new storage technologies to the system. When operating database, the system never directly manipulates database. On the contrary, the system provides a set of CRUD API to perform data retrieving and storing.

6. *Automation Microservice*: The fundamental function is processing events generated by various event sources, analyses them and notifies appropriate participants according to the user specified queries. It supports events filtering, joining, aggregation, grouping, window function etc.

7. *Artifical Intelligence Microservice*: The AI service based on the IoT bigdata, provides a series of Artificial Intelligence tools including machine learning, data mining, graph computing, etc.

8. *Application Microservice*: This service provides components to support various client interaction interfaces, such as web UI, PC client or moblie app etc.

9. *Core service*. The core service provides support for data exchanging by message communication with the devices. The service uses broker to obtain/transform/preprocess messages that produces by objects. Data can be obtained from many sources including COAP, AMQP, WebSockets, MQTT, socket connections, etc. Commands can be sent to devices with various protocols such as MQTT, COAP, and other protocols. More details will be introduced in the following subsection.

3.2 Core microservice specification

Core microservice is a very important microservice in this system, because we use it to handle all functions related to interacting with devices and related microservices. Its responsibilities includes: (i) event management; (ii) plugin management; and (iii) resource discovery. The event manager is responsible for the communication among microservices and devices. Core service is completely based on events, any microservice interaction or any change in the environment will generate events. Events can be trapped by trigger when are published on channel. By defining in automation microservice, each trigger is linked to one or more commands. From this architecture, it follows that the program behavior is not predetermined but is fully modificable at runtime, making it extremely flexible and adaptable to any possible use in building automation.

Core service is provided by the following components, depicted in Fig. 2: BusMsgListener implements a series of bus listener routine which receives message from objects and sends it to its BusConsumer. MessageSubscriber subscribes/unsubscribes a message topic and registers on a channel. BusService provides convenience methods for sending or replying messages on a logic channel. Trigger is a kind of event filter. It listens to the events which subscribes through the channel. If the event satisfies the conditions provided by the trigger, it will invoke BehaviorManager which translates a generic request, one or more logic commands, into a series of hardware commands to ObjAction. Then these commands will be sent to the corresponding actuator automatically.

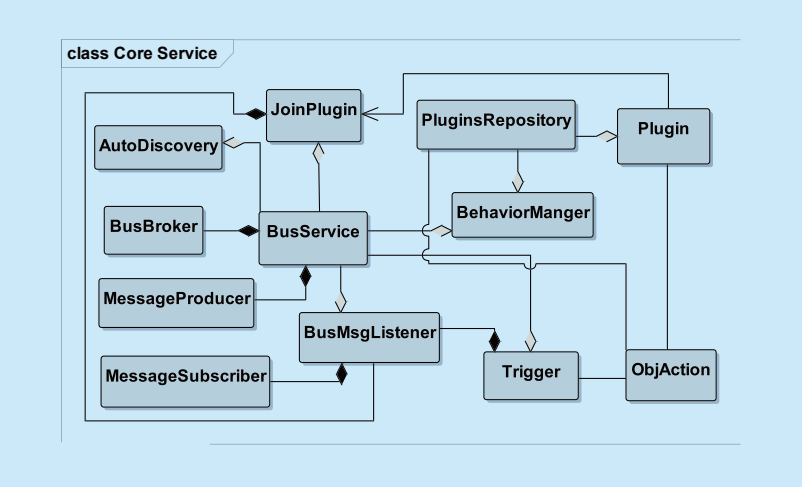

Fig.2 *Core microservice UML class diagram*

The plugin components mount bundles from external device microsevice which registers in PluginsRepository at runtime. All the sensors and actuators supported in device microservice are registered in PluginsRepository.

The resource discovery service is based on the Buservice and AutoDiscovery component. This service is responsible for identifying new nodes in the network and loading the hardware triggers and commands mapping from the metadata retrieved in the device microservice through plugin component.

3.3 Device microservice

This system can manage many heterogeneous devices through extensible plugins. Sensor and actuator plugins are tiny software package which can handle low level device communication and provide API to microservices. Each plugin is managed by device microservice and loaded and initialized automatically at system startup. The communication between the plugin and core microservice is automatically managed through message middleware. Device microservice mainly supports functions as following:

1. Devices registration.

Devices can be manually created by means of API calls or by self-registration. The hardware provides a unique identification to the device service, and then a new device record will be created accordingly as a part of system and will start to accept the new incoming events. In this system, each hardware device should have an only identification in order to be individually addressable. When a device boot for the first time, it connects to the core service network, a registration event will be triggered when establishing the connection with core service. Then the device microservice retrieves corresponding plugins and registers it to core service, in response, core service sends a message to the device service indicating the registration result.

2. Devices events handling

After successful registration, devices can interchange various types of events to core service. These events may include: location updates, temperature, humidity and other data obtaining from the sensors, or alerts. Events are delivered to core service by means of plugins which provides a modular approach to support new functions for manipulating incoming data.

3.Commands delivery

Each hardware managed in a device service has an associated hardware type specification. The specification links to a list of executable commands corresponding to the devices. The commands and parameters can be modified by the administrative console or by REST calls. As the commands execute, a plugin will encode the commands in a predefined format and send them by specified protocol. Fig. 3 illustrates the work process of device service:

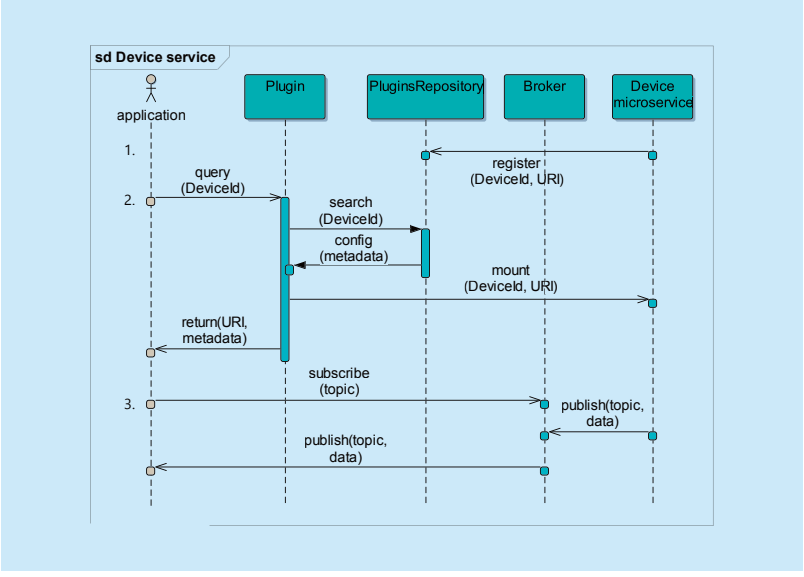

Fig.3 *Device microservice sequence diagram*

Symbols and means

DeviceId: the unique id of the hardware.

URI: the internet address of the device.

Metadata: the metadata which contains the hardware device information.

Data: the data provided and consumed by devices.

Topic: the topic of the Data.

Work process

1. Initialize and register the device

a) Device microservice register (*DeviceId, URI*) to PluginsRepository in Core service.

2. Query device

a) An application sends query to plugin component by (*DeviceId*) .

b) Plugin component searches device plugin registry information in PluginsRepository by (*DeviceId*).

c) PluginsRrepository returns config of the device (*MetaData*).

d) Based on the device metadata, plugin component mounts device plugin with (*DeviceId, URI*).

e) Plugin returns device (*MetaData, URI*) to application.

3. Communication among device and microservice

a) Application begins to subscribe a (*Topic*) on event broker.

b) The device service publishes (*Topic, Data*).

c) When the data meets the specific topic, the application gets (*Topic, Data*).

3.4 Physical layer

As mentioned previously, the device service uses plugins to shield the differences of hardware platform and provides a uniform interface to core service, so the system can support various hardware device platforms, such as Arduino, Raspberry Pi, Andriod and a series of embedded OS. For quick and easy integrating of those popular platforms, the proposed system provides the features as following: device identification, connection orientation, registering new devices, event processing and commands interface. This allows users to focus on building applications rather than struggling with infrastructure. The SDK maintains an internal data structure to represent the environment, objects in it and their state. Making this data available to external clients in a language independent way (XML, JSON, POJOs, ...), is just like that they can see the same environment map the user sees; as a developer you can forget hardware level factors and related issues, and focus on the business process. From this, developers can develop applications in their favorite language, just connecting to the framework and manipulating text messages.

IV. Implementation and Comparison

In the development of this system, we use Docker and Kubernetes as microservice container. The core service is developed on J2EE platform and implemented using Jboss as application server and Resteasy for the creation of REST interface; Asynchronous communication is developed using ActiveMQ. The security services are based on Jasig CAS component. Esper component is used for complex event processing (CEP) and event series analysis in automation service. In AI service, Apache Spark is adopted for data mining and graph computing. GEO service is developed on GeoServer and Openlayers components. Apache Solr is used for SEO application. In Bigdata service, we adopt Redis for photo data storage, kairosDB for time series data storage, MongDB for document storage. As above mentioned, the system supports various kinds of hardware platforms, so it's easy to develop the target software in hardware device system and related plugins in service side system.

Service API Implementation

The system supports REST service which indicates the CRUD operation by standard HTTP verbs as fig. 8.

We demonstrate some REST API call of device service as following:

Create a new device. The hardware needs a unique identification for addressing in system.

Request URI

POST /iotmicservice/devices/

Request header (*brief*):

{

"*devicename*" : "*temperature sensor*"

"*hardwareId*" : "*57d99d89-caab-482a-a0e9-a0a803eed3ba*",

"*plugin*" : "*com.plugins.devices.temperature*",

}

Add trigger for a Device. Measurements

Table I *Microservice IoT framework REST APIs*

REST verb	URI	Parameter	Specification
GET	serviceUrl /objects		Lists resources
	serviceUrl /objects/{Id}	Depends on object function	Gets information about an object
POST	serviceUrl /objects/	hardwareId, device metadata	Creates a new entity
	serviceUrl /objects/{Id}	Measurement, logic, time, signal, etc.	affects some part of the system
PUT	serviceUrl /objects/{Id}	Entity metadata	Updates the addressed entity
DELETE	serviceUrl /objects/{Id}		Deletes the addressed entity

Table II *Comparison results*

Feature	ubiSOAP	Servilla	TinySOA	OpenIoT	Proposed framework
Heterogeneity	Partially support	tinyos	Only tinyos	Support by wrapper	Support by device plugin
Scalability	Application &sensor	Application &sensor Small scale	Application &sensor Small scale	Application &sensor	Microservice &device Large scale
Discovery	device	Device service	Device Service	Device service	Device service
CEP	Not support	Not support	Not support	Partially support	Full support
DevOps	difficult	difficult	hard	difficult	easy

like temperature, humidity, or locations etc. can be added for a device trigger in a request.

Request URI

POST /iotmicservice/devices/{hardwareId}

Request header (*brief*):

{

"*hardwareId*" : "57*d*99*d*89-*caab*-482*a*-*a*0*e*9-*a*0*a*803*eed*3*ba*",

"*temperature*" : [{

"*eventDate*" : "2016-11-21",

"*value*" : 65.0

}],

"*trigger*" :"*alarm*"

}

List events of a device: According to the given criteria, Lists all events that are assigned to the device.

Request URI

GET /iotmicservice/device/{hardwareId}/events

"*results*" : [{

"*eventid*" : "239472398473",

"*eventType*" : "*numerical*",

"*hardwareId*" : "57*d*99*d*89-*caab*-482*a*-*a*0*e*9-*a*0*a*803*eed*3*ba*",

"*value*" : 55}]

To show the benefits of the microservice IoT system, we make the comparison of the current IoT systems in Table 2. From the comparison we can see the benefits of this IoT system that includes more flexibility, scalability, maintainability. It fully supports continuous delivery and development technology.

V. Conclusion & Future Work

In this paper, we propose a new microservice IoT framework that has improved features than existing IoT system such as: monolithic and microserivce system. The proposed system provides a more generic microservice IoT architecture on a module approach, it is much better than existing approaches in terms of flexibility, scalability and platform independency. In addition, we have also introduced the functions of the key components and illustrated their applications. In the proposed microservice IoT framework, all IoT devices and objects are abstracted as resource plugin inside the system.

The allocation of microservices to devices works dynamically by interacting with core service, where the core service interacts with the device by using the related plugins, the device plugins can be mounted or unmounted dynamically, upgradation of the plugins is also possible without affecting the overall system, or even the whole module of a microservice can be freely replaced. Thus, the system not only provides the best IoT service but also has a good support for the Web of Things and has stronger adaptability, scalability and interoperability.

For the practice of microservice architecture, there are still many problems to be studied, such as cooperative transaction processing, network delay, faults in network, message serialization and other issues. Moreover, the application of microservice IoT in machine

learning, search engine optimization and other distributed computing scenarios are also needed to be continuously explored and studied in future.

References

[1] D. Namiot and M. Sneps-Sneppe, "On Micro-services Architecture," *International Journal of Open Information Technologies*, vol. 2, no. 9, pp. 24–27, Sep. 2014.

[2] R. Stephens, *Beginning Software Engineering*, 1st ed. Birmingham, UK, UK: Wrox Press Ltd., 2015.

[3] M. Weyrich and C. Ebert, "Reference Architectures for the Internet of Things," *IEEE Software*, vol. 33, no. 1, pp. 112–116, Jan. 2016.

[4] J. Yick, B. Mukherjee, and D. Ghosal, "Wireless Sensor Network Survey," *Comput. Netw.*, vol. 52, no. 12, pp. 2292–2330, Aug. 2008.

[5] M. Caporuscio, P.-G. Raverdy, and V. Issarny, "ubiSOAP: A Service-Oriented Middleware for Ubiquitous Networking," *IEEE Trans. Serv. Comput.*, vol. 5, no. 1, pp. 86–98, Jan. 2012.

[6] Q. Xiaofeng, L. Wenmao, G. Teng, H. Xinxin, W. Xutao, and C. Pengcheng, "WoT/SDN : web of things architecture using SDN," *China Communications*, vol. 12, no. 11, pp. 1–11, Nov. 2015.

[7] E. Avilés-López and J. A. García-Macías, "TinySOA: a service-oriented architecture for wireless sensor networks," *Service Oriented Computing and Applications*, vol. 3, no. 2, pp. 99–108, Apr. 2009.

[8] Fok, Chien Liang, G. C. Roman, and C. Lu, "Servilla: A flexible service provisioning middleware for heterogeneous sensor networks," *Science of Computer Programming*, vol. 77, no. 6, pp. 663–684, Jun. 2012.

[9] Y. Guo, H. Zhu, and L. Yang, "Service-oriented network virtualization architecture for Internet of Things," *China Communications*, vol. 13, no. 9, pp. 163–172, Sep. 2016.

[10] L. Lan, B. Wang, L. Zhang, R. Shi, and F. Li, "An Event-driven Service-oriented Architecture for Internet of Things Service Execution," *International Journal of Online Engineering*, vol. 11, no. 2, pp. 68–73, Feb. 2015.

[11] J. Soldatos *et al.*, "OpenIoT: Open Source Internet-of-Things in the Cloud," in *Interoperability and Open-Source Solutions for the Internet of Things: International Workshop,* Springer International Publishing, 2015, pp. 13–25.

[12] D. Namiot and M. Sneps-Sneppe, "On Micro-services Architecture," *International Journal of Open Information Technologies*, vol. 2, no. 9, pp. 24–27, Sep. 2014.

[13] T. Vresk and I. Čavrak, "Architecture of an interoperable IoT platform based on microservices," in *2016 39th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, 2016, pp. 1196–1201.

[14] A. Krylovskiy, M. Jahn, and E. Patti, "Designing a Smart City Internet of Things Platform with Microservice Architecture," in *2015 3rd International Conference on Future Internet of Things and Cloud*, 2015, pp. 25–30.

[15] P. Bak, R. Melamed, D. Moshkovich, Y. Nardi, H. Ship, and A. Yaeli, "Location and Context-Based Microservices for Mobile and Internet of Things Workloads," in *2015 IEEE International Conference on Mobile Services*, 2015, pp. 1–8.

Biographies

Long Sun, is currently pursuing the Ph.D. degree in software engineering at College of Computer Science, Sichuan University, Chengdu, China. received the B.S. degree in 1996 from Southwest University, Chongqing, China, and M.S. Degree in 2008 from University of Electronic Science and Technology, Chengdu, China. He get Software Architect qualification issued by The National Qualification Certificate of Computer and Software Technology Proficiency in 2011, he has served on several Natural Science Fund project of national and provincial, among which are the software architecture for the advanced manufacturing technology and the trusted computer system evaluation, statistical natural language processing.

Yan Li, graduated with a B.Eng. in Mechanical Engineering from Sichuan University P. R. China in 1983 and received a M.Sc. degree from the same University in 1986, and received his PhD from Liverpool John Moores University, U.K in 1996. He was a postdoctoral research associate in Cardiff University from 1997 to 1998 and Cambridge University in 1999. In 1988 he was appointed as a lecturer and in 1991 he became an associate professor and in 1999 a professor in Sichuan University. His research interests include application of artificial intelligence techniques in industry, creative product design technology and computer control of manufacturing systems.

Raheel Ahmed Memon, Currently Mr. Memon is pursuing his PhD in the school of Computer Science & Engineering at UESTC, Chengdu, China, in year 2012 he have received his Masters degree in Computer Engineering from Myongji University South Korea. He is a also faculty member at Computer Science Department Sukkur Institute of Business Administration in Sindh, Pakistan. His research interest includes: Internet of Things, Fault tolerant networking, Android Mobile computers and NAND Flash Memories